

Week 9– Robotic design and simulation

ROBOTIC EMBEDDED PROGRAMMING
MENG 4060

ANDY ALUBAIDY, PHD, P.ENG.



WE ARE

HUMBER

Review from Last Week



- **URDF (Unified Robot Description Format)** → XML-based file describing a robot's structure.
- **Links** → Physical components (bodies).
- **Joints** → Connections defining movement between links.
- **Tags** → Define visual, collision, and inertial properties.
- **Extra tags** → Extend function (`material`, `gazebo`, `transmission`).
- **Naming conventions** → Keep files organized and readable.
- **xacro** → Makes URDF modular and efficient:
 - Split into multiple files
 - Reuse code with properties and macros

Agenda for today's training session



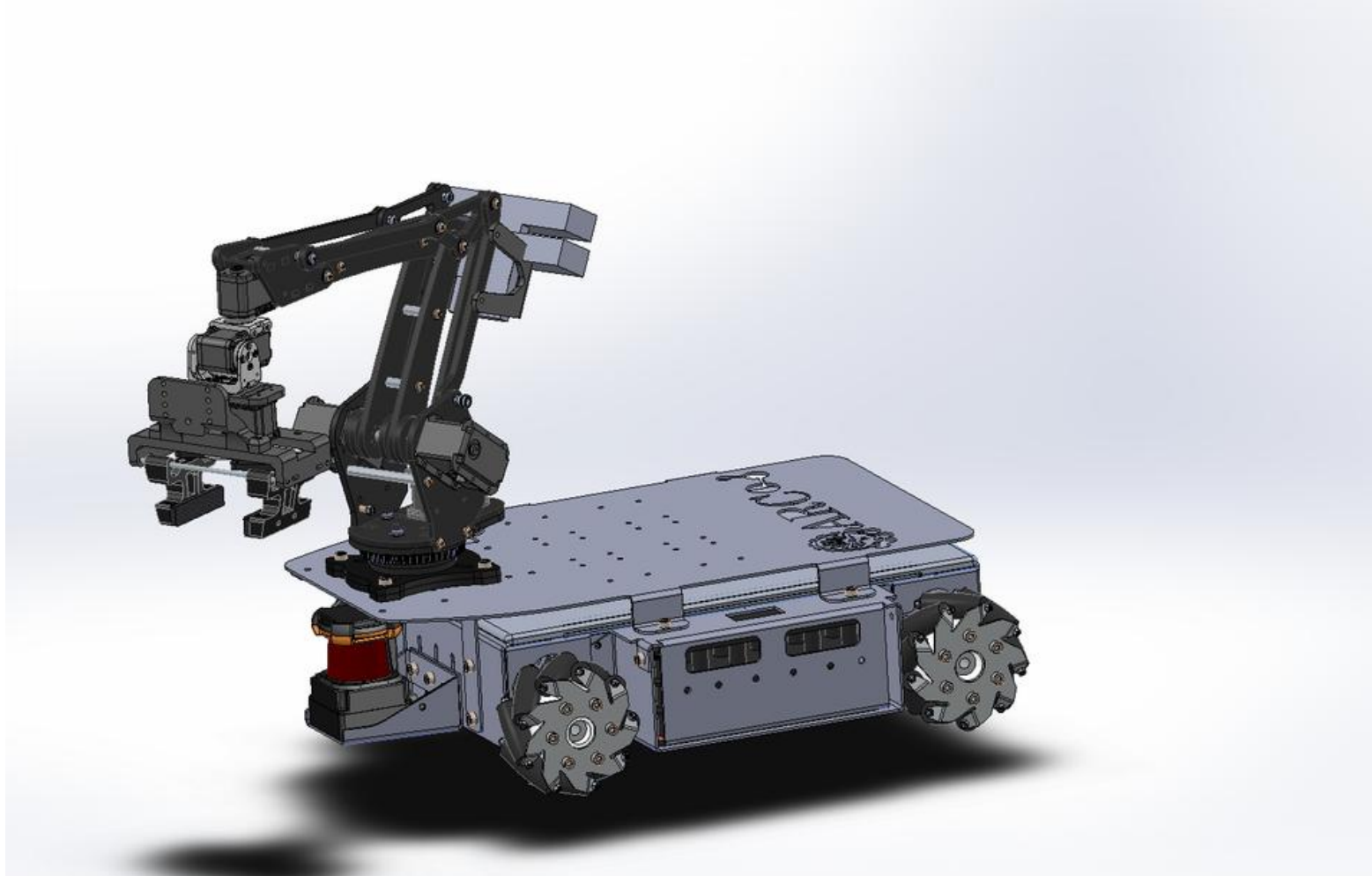
Performance Objective:

- Describe the architecture and core concepts of ROS2 including nodes, publishers/subscribers, services and actions
- Design a complete robotic application using ROS2 to solve open-ended engineering problems.

Problem we will be solving today:

- URDF Design
- Simulation with Gazebo

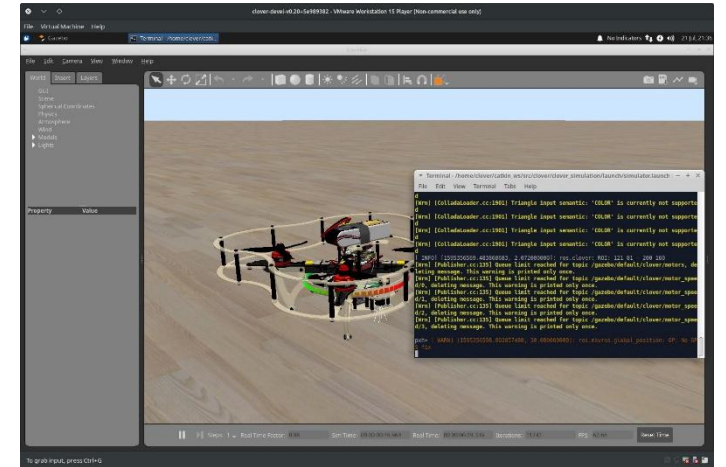
Robotic Design and Simulation: Introduction



[source](#)

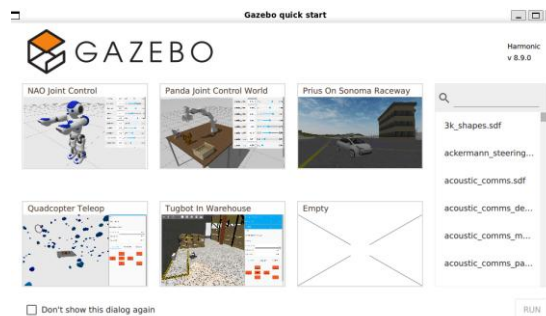
Why Simulation?

- Testing on real hardware can be expensive and risky.
- Early development code can damage physical robots.
- Simulations save time and reduce safety concerns.
- Allows for rapid algorithm development without hardware constraints.



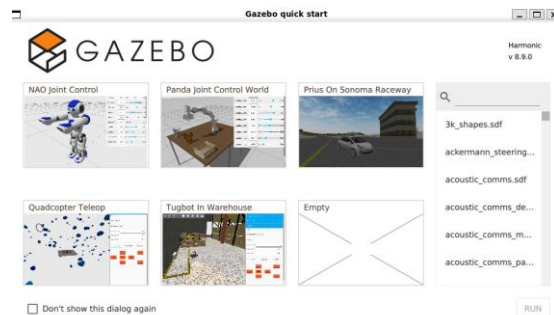
What is Gazebo?

- Gazebo is a **free, open-source** robotics simulation environment.
- Developed by **Open Robotics**, the same organization behind ROS.
- **Not officially part of ROS**, but integrates seamlessly with it.
- Enables realistic testing with simulated robots and environments.



Gazebo vs ROS

- ROS handles robot software architecture (nodes, topics, messages).
- Gazebo focuses on **physics-based simulation** (motion, friction, forces).
- Integration allows Gazebo to publish simulated sensor data as ROS topics.
- Great for testing navigation, control, and perception algorithms.



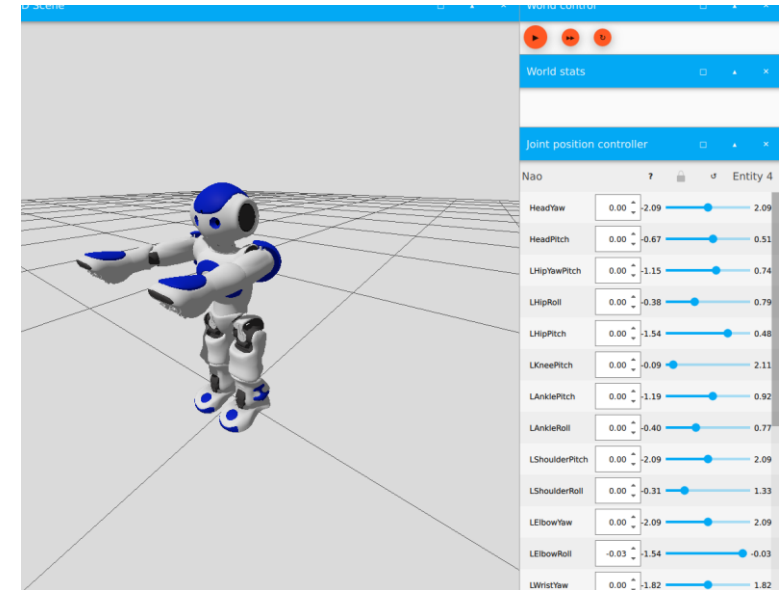
ROS 2

What You Can Do in Gazebo

- Create virtual “worlds” with terrain, lighting, and objects.
- Import or design robot models (URDF/Xacro).
- Simulate sensors (LIDAR, cameras, IMUs).
- Apply realistic physics: **gravity, collision, friction, inertia.**
- Control robots using **ROS 2 nodes.**

Visualization Example

- A sample robot navigating a simulated world.
- Realistic lighting and physics engine.
- Sensors generating live ROS data streams.



Installing Gazebo with ROS 2

- Gazebo integrates directly with ROS 2 via dedicated packages.
- To install with dependencies and plugins:

```
sudo apt install ros-jazzy-gazebo-ros-pkgs
```

- Ensures compatibility between ROS 2 (Jazzy) and Gazebo

```
source /opt/ros/jazzy/setup.bash
```

- Update your system package list and upgrade existing packages by typing:

```
sudo apt-get update && sudo apt-get upgrade
```

- Install **Gazebo Harmonic**, which is the recommended version for ROS2 Jazzy:

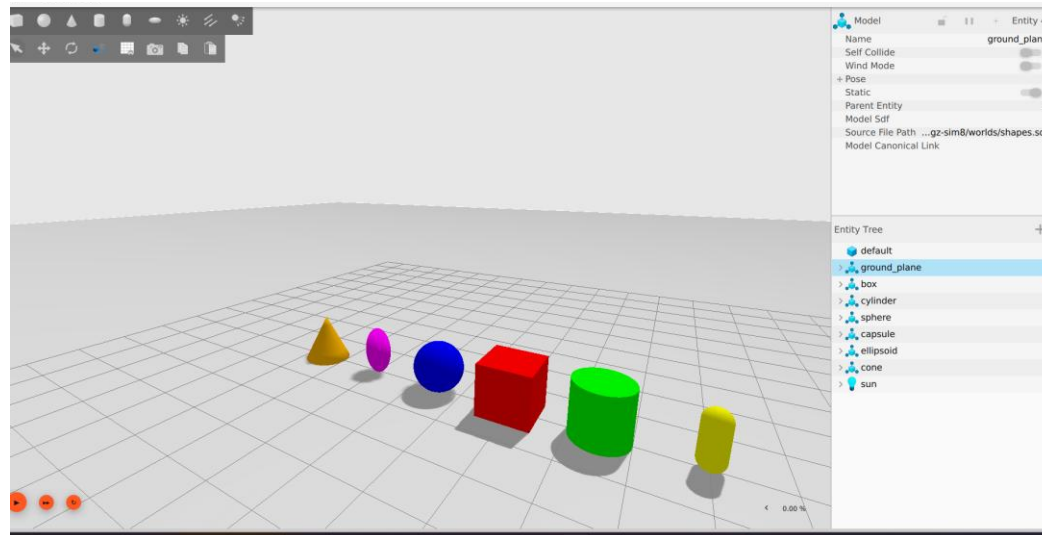
```
sudo apt-get install ros-${ROS_DISTRO}-ros-gz
```

Starting Gazebo

- Launch Gazebo using the updated command:

```
gz sim
```

- To open a specific world file, use the **File** → **Open World** option in the GUI.
- Example: open the **shapes.sdf** world file for testing basic geometry.
- Note: The old command `gazebo /usr/share/gazebo-11/worlds/...` is deprecated in ROS 2 Jazzy.



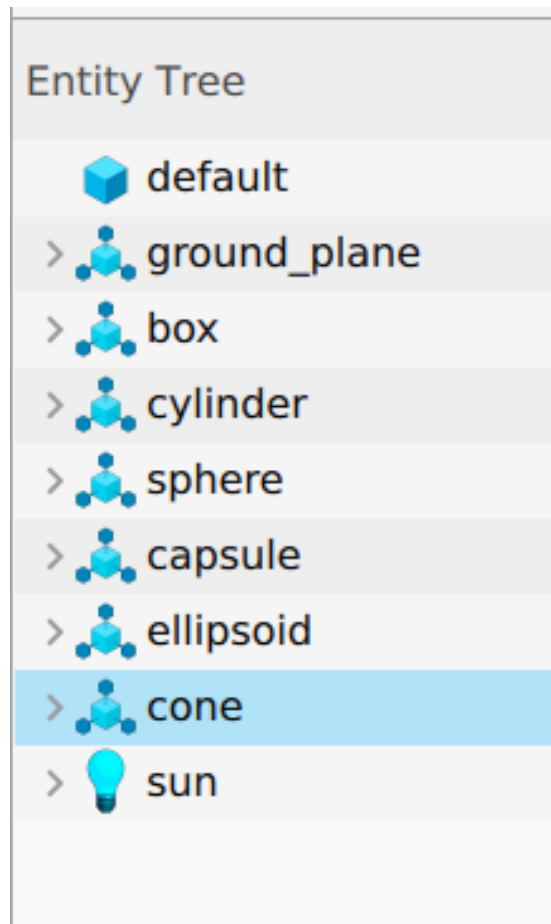
Exploring the Gazebo Interface

- 3D View Controls:
 - Left-click → Pan
 - Middle-click → Orbit
 - Right-click/Scroll → Zoom
- Main Components:
 - World view
 - Model list
 - Physics engine
 - Toolbars for manipulation (translation, rotation, forces)



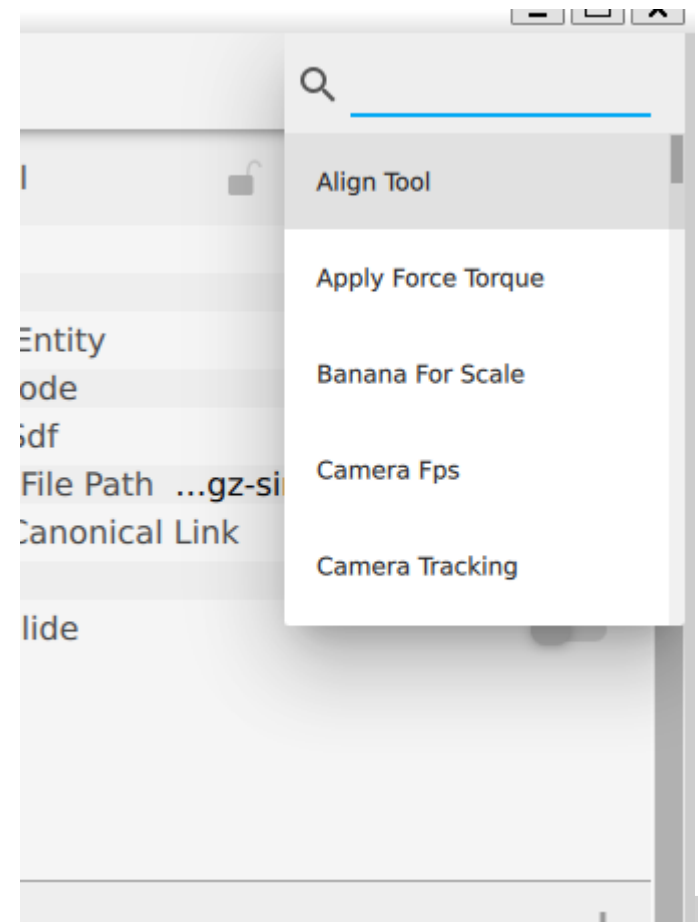
Understanding Gazebo Worlds

- The simulated environment is called a **world**.
- The **Shapes World** demonstrates basic geometric models:
 - Ground plane
 - Box
 - Cylinder
 - Sphere
 - Capsule
 - Ellipsoid
 - Cone
 - Sun (light source)



Manipulating Models

- **Translation Mode:** move models along X/Y/Z axes.
- **Rotation Mode:** rotate models around axes.
- **Apply Force/Torque:** right-click a model → “Apply Force/Torque.”
- **Reset World:** `Ctrl + R` or *Edit* → *Reset World*.
- Restart Gazebo for a full reset.



Hands-On Exercises

Try these activities:

- Move and rotate shapes to explore motion and collisions.
- Pause and resume simulation to test object behavior.
- Modify physical properties (mass, friction) via model editor.
- Apply small forces to observe realistic motion responses.

[Click here: Gazebo-Manipulating Models](#)

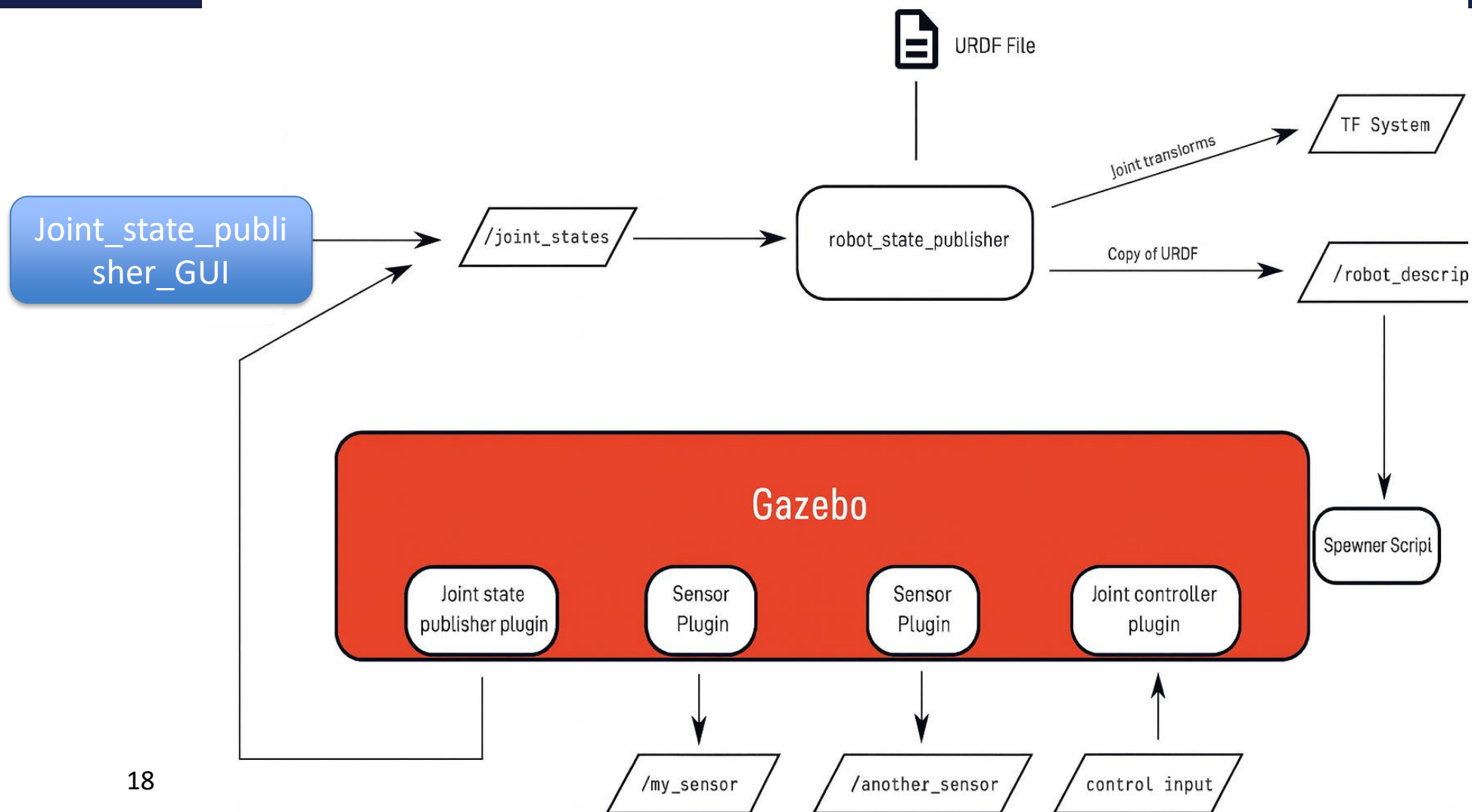
Gazebo and ROS Integration

- Gazebo acts as the **physical simulation environment**.
- ROS 2 provides the **software interface** for robot control and sensing.
- Integration allows ROS 2 nodes to interact with simulated robots as if they were real.
- Compatible with **ROS 2 Jazzy + Gazebo Harmonic** (latest versions).

Fitting Gazebo into ROS 2 Architecture

- Previous setup (without Gazebo):
 - `joint_state_publisher_gui` → Publishes joint angles.
 - `robot_state_publisher` → Broadcasts TFs and publishes URDF.
 - `RViz` → Visualizes robot configuration.
- Adding Gazebo introduces the “real world” simulation layer.
- Gazebo simulates:
 - Motion (via physics engine)
 - Sensors (camera, LIDAR, IMU, etc.)
 - Actuators (motors, joints, grippers)

Fitting Gazebo into ROS 2 Architecture



Gazebo's Role in ROS 2

- Acts as a **virtual testbed** for your robot.
- Replaces the physical hardware layer.
- Receives control commands from ROS 2 topics.
- Publishes simulated sensor data back to ROS 2.
- Enables end-to-end testing of control, navigation, and vision systems.

RDF, SDF, and World Files

- **URDF (Unified Robot Description Format):** describes the robot's structure.
- **SDF (Simulation Description Format):** used natively by Gazebo.
- **World Files:** define the environment containing multiple models.
- Gazebo can automatically convert URDF → SDF using:

```
gz sdf -p model.urdf
```

- Conversion is built into Gazebo Harmonic (works with Jazzy).

File Interactions

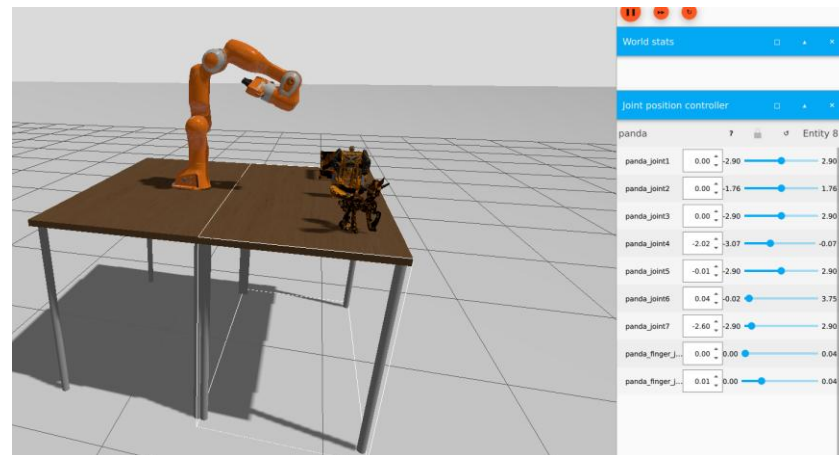
- **URDF:** defines robot geometry, links, joints, and sensors.
- **SDF:** extends URDF to include physics, lights, and materials.
- **World (SDF):** combines models and defines the simulation environment.
- You can spawn robots defined in URDF directly into Gazebo using:

```
ros2 run ros_gz_sim create -file model.urdf -name my_robot
```

- Gazebo automatically handles URDF-to-SDF conversion during spawn.

Why SDF Matters

- Provides richer descriptions: physics, plugins, visuals.
- Allows defining **world-level** parameters (gravity, lights, terrain).
- Enables simulation of multiple robots and environments.
- Supports advanced features like sensors, controllers, and materials.



Gazebo Tags in URDF

- Add Gazebo-specific data inside URDF files using `<gazebo>` tags.
- Used to define simulation properties ignored by other ROS tools.
- Two main forms:
 - **Global:** `<gazebo>` → applies to the entire robot/world.
 - **Referenced:** `<gazebo reference="link_or_joint_name">` → applies to a specific element.
- Gazebo reads these tags to build SDF files during simulation.

Plugins Overview

- ROS 2 and Gazebo use **simulation time** instead of real time.
- Set this parameter for all nodes:

```
ros2 param set /<node_name> use_sim_time true
```

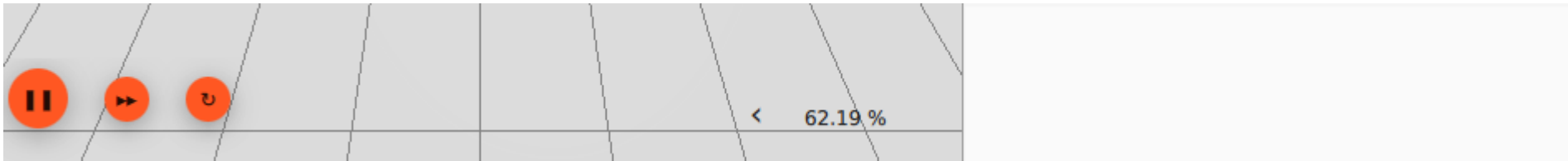
- Launch file example:

```
<node pkg="robot_state_publisher" exec="robot_state_publisher" >  
  <param name="use_sim_time" value="true"/>  
</node>
```

- Ensures all processes stay synchronized with simulation speed.

ROS 2 Jazzy Timing Notes

- **Real time:** based on system clock (Unix time since 1970).
- **Simulation time:** seconds since simulation start.
- Gazebo controls time flow → can pause, speed up, or slow down.
- Each node must explicitly enable `/use_sim_time` (unlike ROS 1 global parameter).



Additional Considerations

- **Naming:** every link and joint must have a unique name.
- **Merging Links:** Gazebo merges URDF links joined by fixed joints into one SDF link.
- **Inertial Properties:** at least one link in every fixed group must include an `<inertial>` tag.
- **World Links:**
 - Fixed robot → root link named `world`.
 - Mobile robot → do **not** name the root link `world`.
- **Materials/Colours:**
 - URDF materials are ignored.
 - Use Gazebo `<material>` tag inside `<gazebo>` for appearance.

Example: Material Tag

```
<gazebo reference="base_link">  
  <material>Gazebo/Blue</material>  
</gazebo>
```

- Ensures proper color rendering in Gazebo Sim.
- Gazebo materials follow naming conventions (e.g., `Gazebo/Red`, `Gazebo/Green`).

Launching Gazebo with ROS 2 Integration

- Previously, Gazebo was started standalone using `gz sim`.
- To integrate Gazebo with ROS 2 Jazzy, use the ROS launch system.
- Example command:

```
ros2 launch ros_gz_sim gz_sim.launch.py world:=src/robot_testing/worlds/world6.sdf
```

- This command:
 - Starts Gazebo Sim (Harmonic) under ROS 2.
 - Optionally loads a specific world file (`.sdf` format).
 - Automatically connects Gazebo topics and services to the ROS 2 ecosystem.

Notes on the Launch File

- The `ros_gz_sim` package replaces `gazebo_ros` from older distributions.
- `gz_sim.launch.py` sets up ROS–Gazebo bridges automatically.
- Make sure your workspace contains the world file you reference.
- Confirm installation with:

```
sudo apt install ros-jazzy-ros-gz-sim
```

Spawning the Robot in Gazebo

- After launching Gazebo, publish the URDF to the `/robot_description` topic.
- Then spawn the robot using the ROS 2 entity spawner tool:

```
ros2 run ros_gz_sim create -topic robot_description -name my_robot
```

- This command:
 - Reads the robot description from `/robot_description`.
 - Creates an entity named **my_robot** in the simulation.
 - Performs automatic URDF → SDF conversion.

ROS-Gazebo Synchronization

- Ensure simulation time is enabled for all nodes:

```
ros2 param set /<node_name> use_sim_time true
```

- Gazebo Sim will publish clock information under `/clock`.
- All ROS 2 nodes using `/use_sim_time=true` stay synchronized with simulation time.

Using a Custom Launch File

- Combine all essential nodes into one launch file for automation.
- Typical launch setup includes:
 - `robot_state_publisher` (with `use_sim_time=true`)
 - `ros_gz_sim` (Gazebo launcher)
 - Robot spawner script (`create` node)

Materials and Colours in Gazebo

- URDF `material` tags don't appear correctly in Gazebo Sim.
- Instead, define colour through Gazebo tags:

```
<gazebo reference="my_link">  
  <material>Gazebo/Black</material>  
</gazebo>
```

- Valid examples: `Gazebo/Red`, `Gazebo/Blue`, `Gazebo/Chrome`, etc.

Publishing Joint States and Controllers

- Gazebo must simulate actuators and publish joint data.
- Two core tasks:
 1. Receive control commands.
 2. Publish joint states on `/joint_states`.
- Best practice: use `gazebo_ros2_control` plugin (part of `ros2_control`).
- Allows reusing controllers for both simulated and real robots.

Simpler Plugins Example

Joint State Publisher Plugin:

```
<plugin name="gazebo_ros_joint_state_publisher"
        filename="libgazebo_ros_joint_state_publisher.so">
  <update_rate>20</update_rate>
  <joint_name>slider_joint</joint_name>
  <joint_name>arm_joint</joint_name>
</plugin>
```

Simpler Plugins Example

Joint Control Plugin:

```
<gazebo>  
  <plugin name="gazebo_ros_joint_pose_trajectory"  
    filename="libgazebo_ros_joint_pose_trajectory.so">  
    <update_rate>20</update_rate>  
  </plugin>  
</gazebo>
```

Sending Joint Commands

- Once spawned and configured, control joints using topics:

```
ros2 topic pub -1 /set_joint_trajectory trajectory_msgs/msg/JointTrajectory \
'{header: {frame_id: world}, joint_names: [slider_joint, arm_joint], points: [ {positions: [0.8, 0.6]} ]}'
```

- The robot should update its pose in both Gazebo and RViz.
- If unstable, add damping and friction to joints:

```
<dynamics damping="10.0" friction="10.0"/>
```

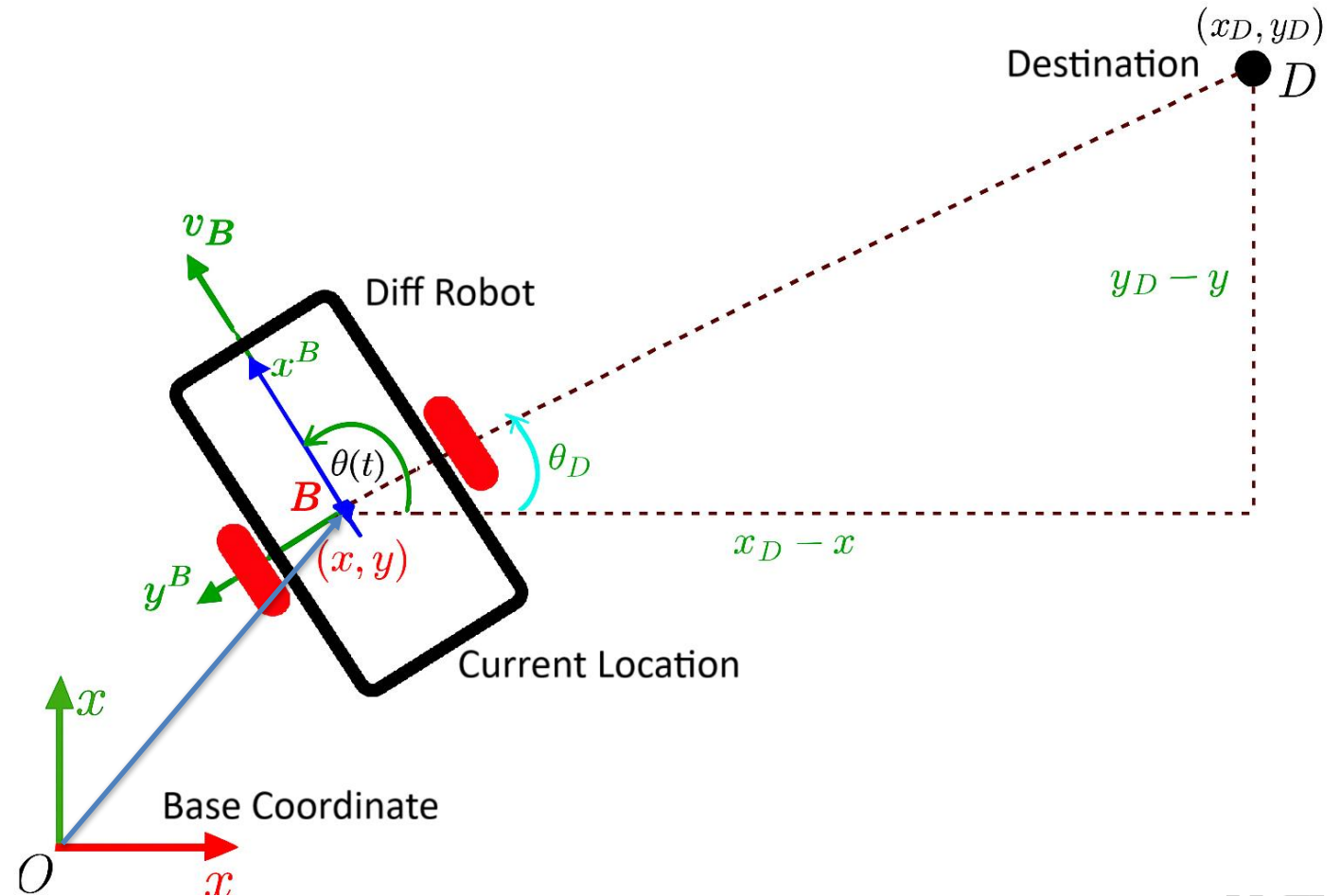
Adding Sensors to URDF

- Sensors are defined in `<sensor>` blocks inside `<gazebo>` references.
- Each sensor must attach to a link and specify a type and plugin.
- Example: LIDAR Sensor (Ray Type)

```
<gazebo reference="lidar_link">
  <sensor name="lidar" type="ray">
    <ray>
      <!-- Ray-specific parameters -->
    </ray>
    <plugin name="laser_controller" filename="libgazebo_ros_ray_sensor.so">
      <!-- ROS 2 LIDAR plugin parameters -->
    </plugin>
  </sensor>
</gazebo>
```

Position Controller Description (Differential Drive Robot)

- Robot center $\mathbf{B}(x, y)$ moves in global frame $\mathbf{O}(x, y)$ with orientation $\theta(t)$.
- Destination $\mathbf{D}(x^D, y^D)$ is the desired final position.
- Objective: steer $\mathbf{B} \rightarrow \mathbf{D}$ by controlling linear velocity $\mathbf{v}^B$ and angular velocity θ .



Mathematical Model - Desired Orientation Angle

The angle between the line **BD** and the x-axis:

$$\tan(\theta_D) = \frac{y_D - y}{x_D - x}$$

Equivalent form using the two-argument arctangent:

$$\theta_D = \text{atan2}(y_D - y, x_D - x)$$

Mathematical Model - Angular Velocity Control

Proportional controller driving the robot to align with θ^D :

$$\dot{\theta}_c = K_\theta(\theta_D - \theta)$$

- K_θ : angular control gain.
- As $\theta \rightarrow \theta^D$, control action $\rightarrow 0$ ensuring stability.

Distance and Linear Velocity Control

Distance between current and target position:

$$d_{BD} = \sqrt{(x_D - x)^2 + (y_D - y)^2}$$

Linear velocity proportional to distance:

$$v_c = K_v \sqrt{(x_D - x)^2 + (y_D - y)^2}$$

- K_v : translational gain controlling approach speed.

Combined Control Laws

Simplified proportional control model:

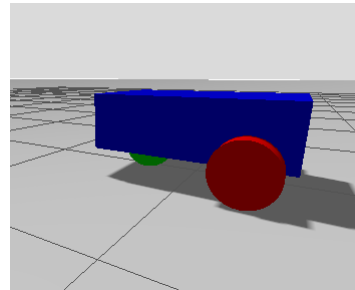
$$\begin{cases} \theta_D = \text{atan2}(y_D - y, x_D - x) \\ \dot{\theta}_c = K_\theta(\theta_D - \theta) \\ v_c = K_v \sqrt{(x_D - x)^2 + (y_D - y)^2} \end{cases}$$

Controller Behavior and Insights

- **Angular Controller:** aligns robot with direction to target.
- **Linear Controller:** drives robot forward proportional to distance.
- As $d^{BD} \rightarrow 0$, both v_c and $\theta_c \rightarrow 0$ ensuring convergence.
- Proportional-only design; can extend to **PID** for precision.

Class Activity: Design of Diff Robot and Simulate in Gazebo

- **Step 1: Design the Robot (Mechanical Design)**
- **Step 2: Create URDF Model**
- **Step 3: Add Gazebo-Specific Xacro**
- **Step 4: Create Launch Files**
- **Step 5: Modify Build Configuration**
- **Step 6: Create Parameter & Config Files**
- **Step 7: Simulate in Gazebo**
- **Note: We will discuss the Electrical part next week**



[source](#)

Class Activity: Design of Diff Robot

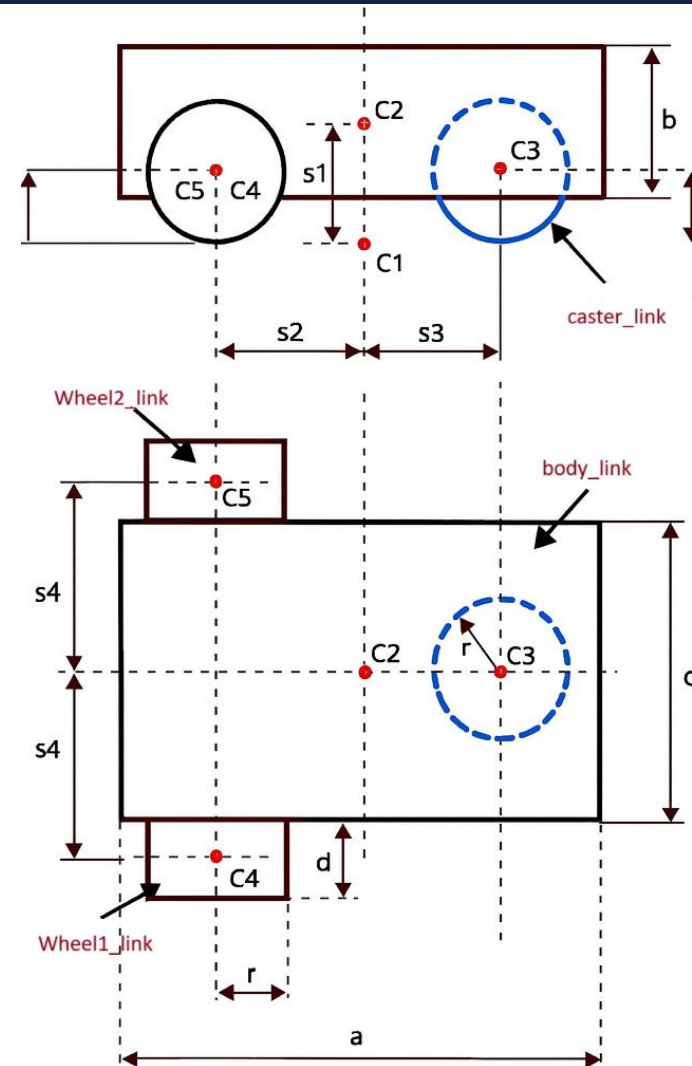
- **Body_link:**

- **Volume:** $V_1 = a \cdot b \cdot c$
- **d1=**density
- **Mass:** $m_1 = d_1 \cdot V_1 = d_1 \cdot a \cdot b \cdot c$
- **moments of inertia:**

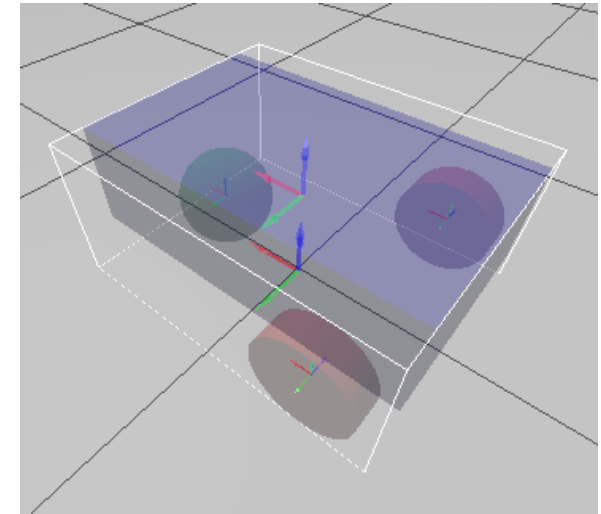
$$I_x = \frac{1}{12} m_1 (b^2 + c^2)$$

$$I_y = \frac{1}{12} m_1 (a^2 + b^2)$$

$$I_z = \frac{1}{12} m_1 (a^2 + c^2)$$



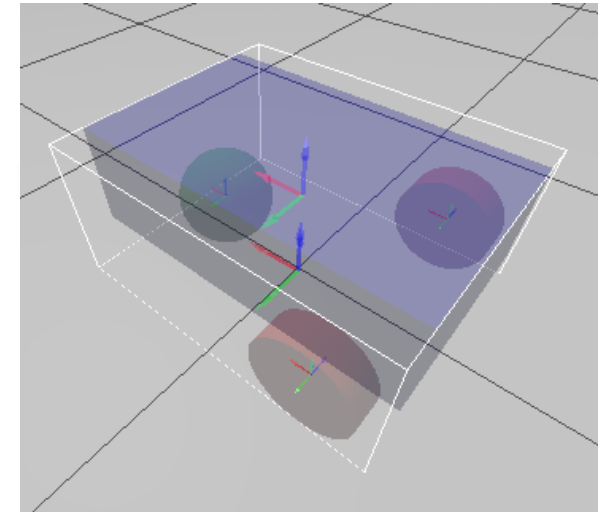
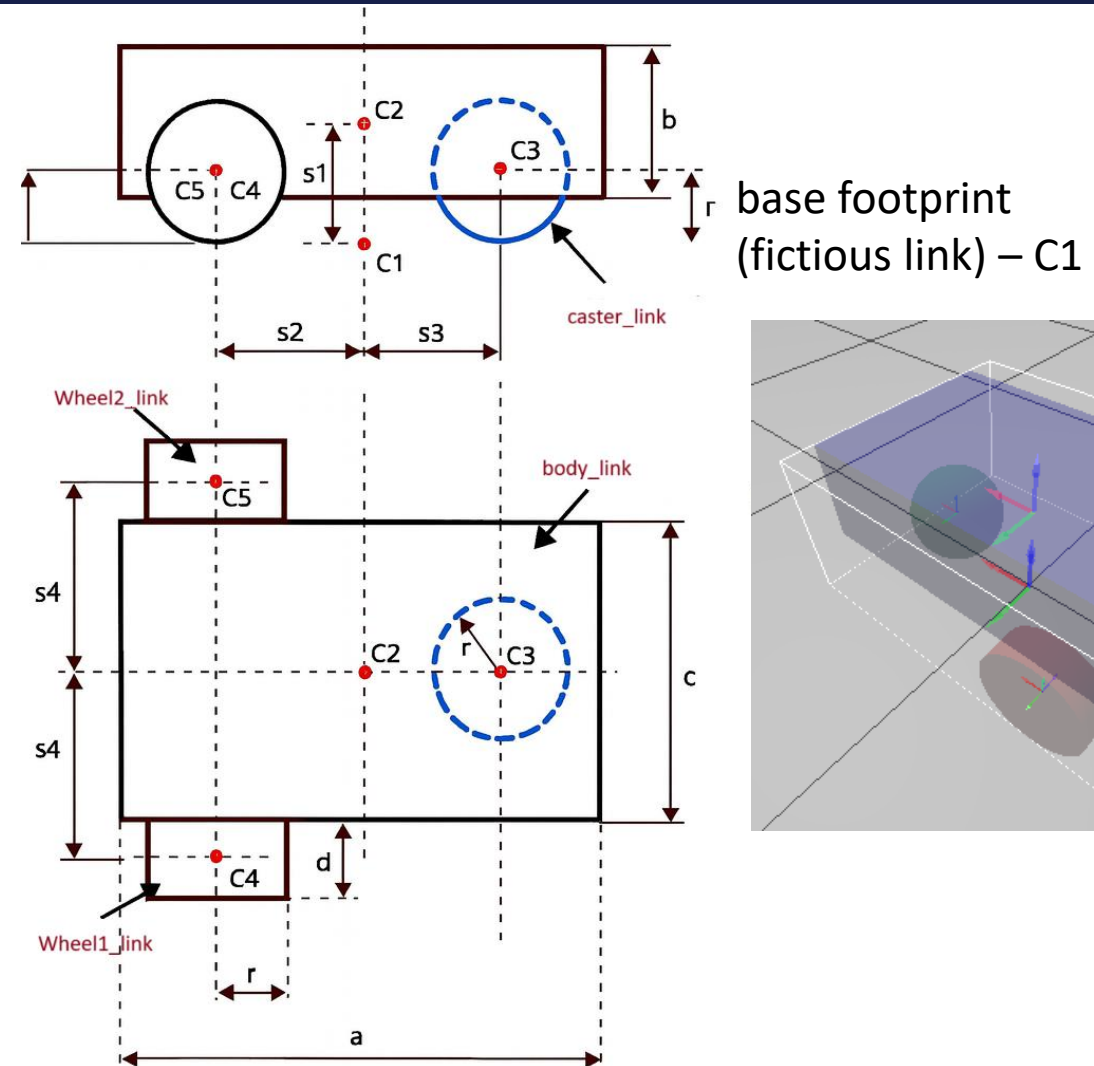
base footprint
(fictitious link) – C1



Class Activity: Design of Diff Robot

- **Wheel1_link & Wheel2_link:**
 - **Volume:** $V2 = \pi \cdot r^2 \cdot d$
 - **d2=density**
 - **Mass:** $m2 = d2 \cdot V2 = d2 \cdot \pi \cdot r^2 \cdot d$
 - **moments of inertia:**

$$\begin{aligned} I_x &= \frac{1}{12} m_2 (3r^2 + d^2) \\ I_y &= \frac{1}{12} m_2 (3r^2 + d^2) \\ I_z &= \frac{1}{2} m_2 r^2 \end{aligned}$$



Class Activity: Design of Diff Robot

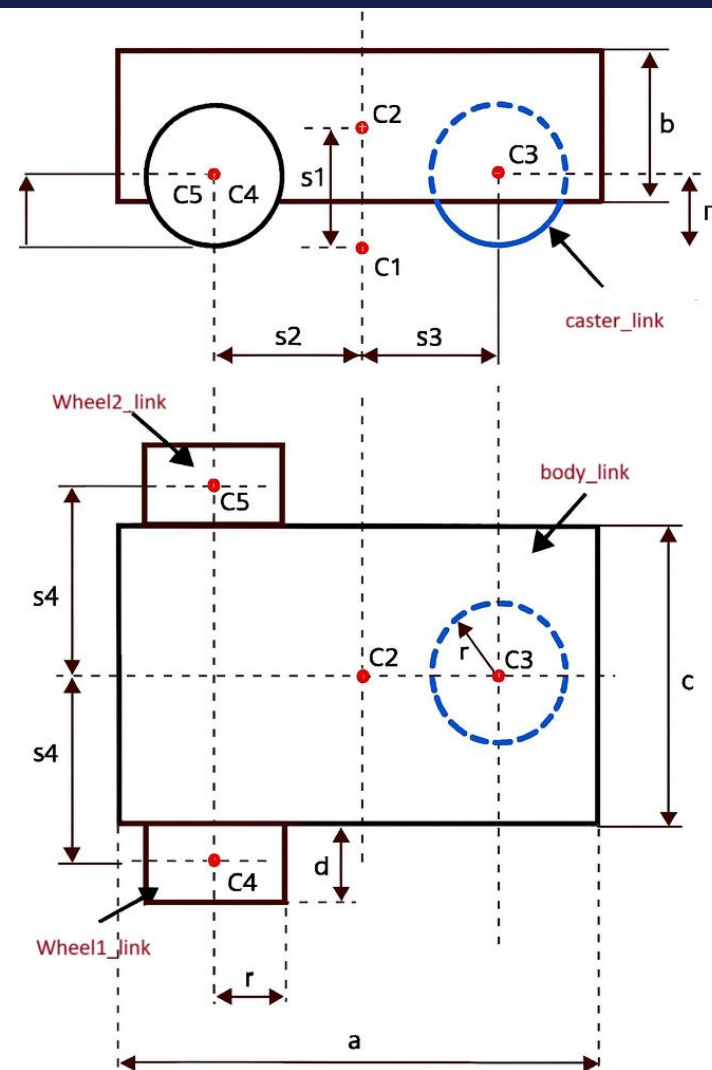
- caster_link:**

- Volume: $V3 = 4/3 \cdot \pi \cdot r^3$
- d3=density
- Mass: $m3 = d3 \cdot V3 = d3 \cdot 4/3 \cdot \pi \cdot r^3$
- moments of inertia:

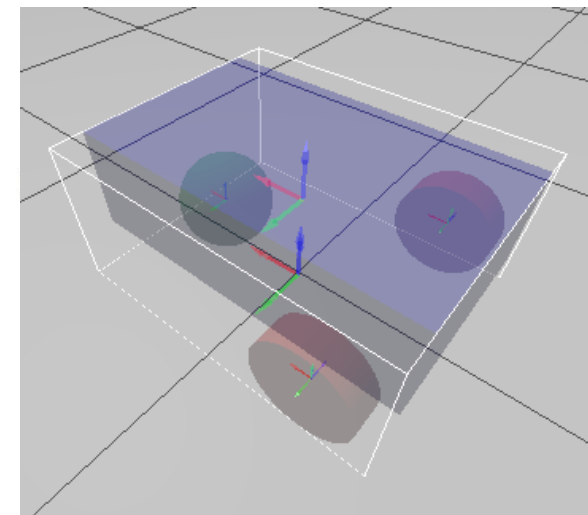
$$I_x = \frac{2}{5} m_3 r^2$$

$$I_y = \frac{2}{5} m_3 r^2$$

$$I_z = \frac{2}{5} m_3 r^2$$



base footprint
(fictitious link) – C1



Class Activity: Design of Diff Robot – Packages to install

```
source /opt/ros/jazzy/setup.bash
```

```
sudo apt update && sudo apt upgrade -y
```

```
sudo apt-get install ros-${ROS_DISTRO}-ros-gz
```

```
which gz
```

```
gz sim (this will launch Gazebo, close it for now)
```

```
sudo apt-get install gedit
```

```
sudo apt install ros-jazzy-joint-state-publisher
```

```
sudo apt install ros-jazzy-joint-state-publisher-gui
```

```
sudo apt install ros-jazzy-xacro
```

```
sudo apt-get install ros-jazzy-teleop-twist-keyboard
```

```
ros2 pkg list | grep gz
```

Class Activity: Design of Diff Robot – create workspace

```
source /opt/ros/jazzy/setup.bash
```

```
sudo apt update && sudo apt upgrade -y
```

```
cd ~
```

```
mkdir -p lec9_ws/src
```

```
cd ~/lec9_ws
```

```
colcon build
```

```
cd ~/lec9_ws/src
```

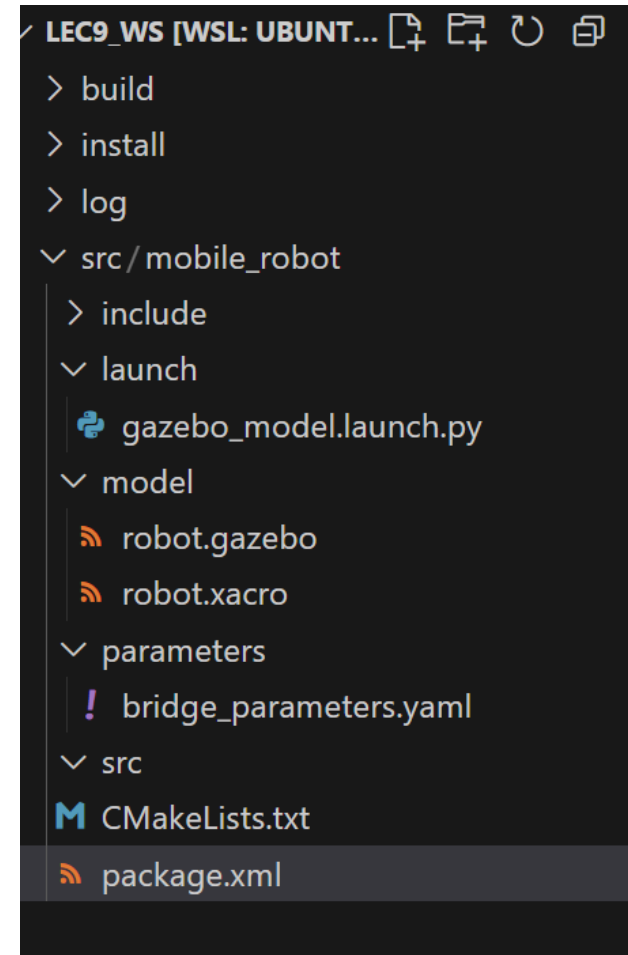
```
ros2 pkg create --build-type ament_cmake mobile_robot
```

```
cd ~/lec9_ws/
```

```
code . (Launch VS)
```

Class Activity: URDF, Gazebo, and Parameter Files

- ✓ Create three folders in VS under the package: **launch, model, parameters**
- ✓ Under 'model' folder create: **robot.xacro**
- ✓ Copy paste the contents of the file (from BB)
- ✓ Under 'model' folder create: another file **robot.gazebo**
- ✓ Copy paste the contents of the file (from BB)
- ✓ Under 'parameters' folder create: **bridge_parameters.yaml**
- ✓ Copy paste the contents of the file (from BB)
- ✓ Under 'launch' folder create: **gazebo_model.launch.py**
- ✓ Copy paste the contents of the file (from BB)



Class Activity: Design of Diff Robot – Adjustment

- Open package.xml and add the following before `<export>`:

```
<exec_depend>joint_state_publisher</exec_depend>
```

```
<exec_depend>robot_state_publisher</exec_depend>
```

```
<exec_depend>gazebo_ros</exec_depend>
```

```
<exec_depend>xacro</exec_depend>
```

```
<exec_depend>ros_gz_bridge</exec_depend>
```

- Open CMakeLists.txt file and add the following before “if(BUILD_TESTING)”:

```
install(
```

```
  DIRECTORY launch model parameters
```

```
  DESTINATION share/${PROJECT_NAME}
```

```
)
```

Class Activity: Design of Diff Robot – launch

Back to CL terminal:

```
source /opt/ros/jazzy/setup.bash
```

```
cd ~/lec9_ws
```

```
colcon build
```

```
source ~/lec9_ws/install/setup.bash
```

```
ros2 launch mobile_robot gazebo_model.launch.py
```

Class Activity: Design of Diff Robot – Control the robot

Open a new terminal (do not close the other terminal)

```
source /opt/ros/jazzy/setup.bash
```

```
ros2 run teleop_twist_keyboard teleop_twist_keyboard
```

Class Activity: Design of Diff Robot – Observe

Open a new terminal (do not close the other terminal)

```
source /opt/ros/jazzy/setup.bash
```

```
ros2 topic list
```

What do you notice?

```
ros2 topic echo /odom
```

What do you notice?

```
ros2 topic info /cmd_vel
```

Challenge: Change the colors

Summary



- Gazebo Sim (Harmonic) provides realistic, physics-based environments for testing robots safely.
- Integrated with ROS 2 Jazzy using the `ros_gz_sim` bridge for full sensor and actuator interaction.
- Use **launch files** to automate startup: `robot_state_publisher`, `ros_gz_sim`, and robot spawner.
- Extend URDF with `<gazebo>` tags and **plugins** for control, sensors, and visuals.
- Always enable `use_sim_time` for synchronized simulation.
- Customize visuals with Gazebo materials (e.g., `Gazebo/Blue`).
- Add sensors like LIDAR or cameras using `<sensor>` and plugin tags.
- **Key takeaway:** Gazebo + ROS 2 enable full virtual prototyping—build, test, and control robots in simulation before deploying to hardware.

THANK YOU.



WE ARE

HUMBER